Backend – szerveroldali működés

1. A backend feladata

A backend a weboldal szerveroldali része. Feladata, hogy fogadja a felületről érkező kéréseket, ellenőrizze a beküldött adatokat, elvégezze a szükséges műveleteket, majd válasszal térjen vissza. Így az adatok kezelése nem kizárólag a böngészőben történik, hanem a szerveren keresztül, az adatbázissal együttműködve.
A projekt backendje Node.js környezetben fut, az Express keretrendszert használja. Az Express segítségével külön végpontok rendelhetők az egyes műveletekhez. A szerver a kérések törzsében érkező JSON-adatokat az express.json() middleware-rel olvassa be, a válaszokat pedig szintén jellemzően JSON-formátumban küldi vissza.

2. Technológiák és adatbázis-kapcsolat

A szerver Node.js és Express használatával készült. A MySQL-adatbázissal a mysql2 csomag kapcsolja össze a programot, annak Promise-alapú változatán keresztül. A kapcsolat beállításai külön db.js fájlban találhatók, így az adatbázis elérése elkülönül a végpontokat kezelő kódtól.
A kapcsolat adatai – például a kiszolgáló címe, a felhasználónév, a jelszó és az adatbázis neve – környezeti változókból is megadhatók. A program alapértelmezett beállításokat is tartalmaz a helyi fejlesztéshez. A mysql2 kapcsolatkészlete lehetővé teszi, hogy a szerver több adatbázis-műveletet kezeljen anélkül, hogy minden kérésnél külön kapcsolatot kellene létrehozni.
A SQL-lekérdezések paraméterezett formában szerepelnek. A beküldött értékek külön paraméterként kerülnek a lekérdezésekbe, ami biztonságosabb megoldás, mint az értékek közvetlen összefűzése az SQL-parancsokkal.

3. API-végpontok és adatműveletek

A backend REST API-szerű felépítést követ. Az API-végpontok segítségével a kliensprogram egységes módon kérhet le vagy módosíthat adatokat. A megvalósított végpontok négy fő adatkört kezelnek: diákok, tanárok, értékelések és időpontok.
A GET kérések az adatok lekérésére szolgálnak: a program képes egy adott adatkör rekordjait listázni, illetve azonosító alapján egyetlen rekordot lekérni. A POST kérések új rekordot hoznak létre. A PUT kérések egy meglévő rekord adatainak módosítását végzik, a DELETE kérések pedig törlik a megadott rekordot. Sikeres

létrehozáskor a szerver 201-es státuszkódot ad vissza, sikeres törléskor pedig 204-es választ is használ.
A diákoknál a név és az e-mail-cím, a tanároknál az értékeléshez tartozó azonosító, a név és az e-mail-cím kezelhető. Az értékelésekhez diákazonosító, 1 és 5 közötti egész pontszám, valamint szöveges vélemény tartozik. Az időpontoknál a tanár és a diák azonosítója, a kezdési és befejezési idő, a dátum, valamint az óra típusa szerepel. Az időpont típusánál a program az „online", „jelenléti" és „feladatmegoldás" értékeket fogadja el.

4. Adatellenőrzés

A szerver a műveletek végrehajtása előtt ellenőrzi a beérkező adatokat. A szöveges mezőknél vizsgálja, hogy az érték valóban szöveg-e, nem üres-e, és nem hosszabb-e a megengedettnél. Az e-mail-címeknél alapvető formátumellenőrzés történik. Az azonosítóknak pozitív egész számoknak kell lenniük.
Az időpontoknál a backend ellenőrzi az óra és perc formátumát, a dátum helyességét, valamint azt, hogy a kezdési idő korábbi legyen a befejezési időnél. Az értékelés pontszámának 1 és 5 közé kell esnie, az időpont típusa pedig csak az előre meghatározott értékek egyike lehet. Ha az adatok hiányosak vagy hibásak, a szerver nem hajtja végre a műveletet, hanem 400-as státuszkódú hibaüzenetet küld.
Az azonosítókat egy közös ellenőrzés is vizsgálja, mielőtt az adott végpont feldolgozná a kérést. Módosításkor a program ellenőrzi, hogy a módosítandó rekord létezik-e. Ha nem található, 404-es státuszkódot és hibaüzenetet küld vissza.

5. Hibakezelés és állapotellenőrzés

A backend külön kezeli a nem létező API-végpontokat. Ezekre 404-es státuszkóddal és rövid, JSON-formátumú hibaüzenettel válaszol. A hibás JSON-tartalom 400-as választ eredményez. Az adatbázisban előforduló hivatkozási hibáknál a szerver jelzi, ha egy művelet nem létező kapcsolódó rekordra hivatkozik, vagy egy rekord azért nem törölhető, mert más adatok kapcsolódnak hozzá.
Az általános, nem külön kezelt hibák esetén a backend 500-as státuszkódot küld, és szerverhibát jelez. A /api/health végpont egy egyszerű adatbázis-lekérdezéssel ellenőrzi, hogy a szerver eléri-e az adatbázist. Sikeres ellenőrzés esetén a válasz jelzi, hogy a szolgáltatás és az adatbázis elérhető.